Lodos v2: Reading Guide

Every source file, and every kernel one step launches, in order

Lodos v2 as of 2026-11-04, for someone about to read the code. The kernel lists were recorded from captured steps (Section 2). Why each part is as it is: the Development Guide (`docs/development_guide.pdf`) and the two manuals.

---

1 The layout, and where to start

Lodos v2 is two layers. `lodos/` is the simulator: rigid-body dynamics (`rbd`), collision (`col`), contact dynamics (`cd`), the step that runs them, the MJCF model on the device (`mjcf`) and the helpers they share (`utils`). It carries the parts of MuJoCo Warp (MJWarp) it runs as its own code, adapted from MuJoCo Warp 3.14.0: the collision Lodos runs on the MJCF's geoms (`col/prim`), the constraint solve it runs on every contact, with a proximal loop around the Newton solve for the exact mode (`cd/solver`), and the model and data the two share (`mjcf`). `viewer/` is the live viewer, with the scene builders and their assets; the simulator imports nothing from it. The benchmarks and tools are in Section 3.1. In the order a step runs the code:

1. `lodos/sim.py`: `make_sim` builds every context of a scene once; its docstring is the overview.
2. `lodos/step/__init__.py`: the step, and the CUDA graph that captures it.
3. `lodos/rbd/forward.py`, then `smooth.py`: the rigid-body pass.
4. `lodos/col/prim/driver.py`: MJWarp's collision on Lodos's poses, with the exact hull narrowphase.
5. `lodos/col/sdf/collide.py`, and `make_collision_context`'s docstring in `types.py`: Lodos's collision.
6. `lodos/cd/driver.py`: Lodos's contacts in MJWarp's solve; `resolve_options` documents every option.
7. `lodos/cd/solver/constraint.py`, then `nlsp.py`: the solve itself, its rows, then Newton's method and the proximal loop around it.
8. `viewer/session.py` and `viewer/scenes/catalog.py`: how the viewer builds and runs a scene.

2 One step, kernel by kernel

`lodos.step.step_sim(sim)` is `forward(sim)` — the rigid-body pass, MJWarp's collision, Lodos's collision and the contact solve — and then the integrator; `make_step_graph` captures it into one CUDA graph. Table 1 lists every kernel it launches, in order, as recorded by wrapping Warp's `launch` during that capture: the default options (the exact mode) on `arm_box_6`, a scene with both pipelines, at one world, 361 launches, where a loop body counts once; besides them the step calls CUB twice, a scan (row 19) and a sort (row 35). L is the depth of the kinematic trees: a level-by-level kernel launches once a level. Rows in brackets launch only for the scenes or options they name; Table 2 gathers the options that add or replace kernels.

Table 1: The kernels of one step, in launch order. File paths are under `lodos/`.

#	call	kernels	what they do
	Rigid-body pass	`rbd/forward.py`: `fwd_rbd`	
1	`smooth.kinematics`	`_kinematics_level` $\times L$ [`_site_poses`]	each body's world pose and its joints' anchors and axes, a launch a tree level; [the sites' frames]
2	`smooth.com_pos`	`_subtree_init`, `_subtree_acc` $\times L$, `_subtree_div`, `_cinert`, `_cdof`	subtree centres of mass; each body's inertia about its root's centre of mass; each dof's motion axis
3	`smooth.build_m`	`_crb_acc` $\times L$, `_build_M` [`_zero_big`, `_build_M_rows`]	composite rigid-body inertias, then each tree's block of M with armature; [trees of 13 dofs or more: a dof a thread]

#	call	kernels	what they do
4	`smooth.factor_blocks`	`_factor_M` [`_gather_big`, tile `factor`, `_scatter_big`]	$M = LL^\top$, one thread a tree; [large trees: Warp's tile Cholesky]
5	[actuators] `smooth.transmission, actuator_velocity`	`_length`, `_moment`, `_actuator_velocity`	each actuator's length, its moment $\partial l/\partial \dot{q}$, and its rate
6	`smooth.com_vel`	`_com_vel_level` $\times L$	body velocities, and the motion axes' rates
7	`passive.passive`	`_spring_damper`	joint springs and dof damping, and gravity compensation
8	`smooth.rne`	`_cacc_level` $\times L$, `_cfrc_local`, `_cfrc_acc` $\times L$, `_qfrc_bias`	Newton–Euler at $\ddot{q} = 0$: bias accelerations, gravity as the world's; body forces summed up the trees, on the dofs
9	[actuators] `forward.actuation`	`_actuator_force`, `_qfrc_actuator`	each actuator's force from its clamped control, on the dofs
10	`forward.fwd_acceleration`	`_qfrc_smooth`	every non-constraint force, summed
11	`smooth.solve_m`	`_solve_qacc_smooth` [`_gather_rhs_big`, tile `solve`, `_scatter_qacc_big`]	$\ddot{q}_{\text{smooth}} = M^{-1} f_{\text{smooth}}$ from the factor
12	[implicitfast] `implicit.prepare`	`_factor_M`	the factor of $M - hD$, for the integrator
	MJWarp's collision	`col/prim/driver.py`: `PrimCollision`	where the MJCF's geoms have pairs to collide
13	`poses`	`geom_kinematics`	each pairable geom's world pose, from its body's
14	`collision_driver.nxn_broadphase`	`_nxn_broadphase`	MJWarp's candidate pairs, tested by bounding volumes; the overlapping ones queued
15	`collision_primitive.primitive_narrowphase`	`_primitive_narrowphase`	analytic contacts of primitive pairs (planes, spheres, capsules, boxes), into `d.contact`
16	`hybrid/pipeline.py: launch`	`clear_cache_kernel`, `warm_kernel`, `gjk_kernel`, `epa_warp_kernel` (`epa_kernel` at large batches), `manifold_kernel`, `plane_kernel`, `flip_cache_kernel`	hull pairs (meshes, boxes) and plane–mesh, exactly: last step's features re-checked by a certificate; GJK on the rest; EPA on the penetrating, decoded into hull features; a quadrangle-clipped manifold; the floor's contacts; the cache's halves swapped
17	`collide`	`export_kernel`	the hybrid's contacts appended to `d.contact`
	Lodos's collision	`col/sdf/collide.py`: `collide`	where there are Lodos pairs
18	`broadphase_bbox`	`compute_T_w`, `compute_T_rel`, `obb_filter`, `gen_weights`, `tile_filter`	world poses; each pair's vertex body in its SDF body's frame; a 15-axis SAT between the pair's boxes; tiles per generator; a box test a tile
19	`compact_nTa`	CUB's scan, `set_ct_nTa`, `scatter_nTa`, `build_ul_nCa1_tiled`	the surviving tiles compacted, by a scan over their generators, and each segment's bounds
20	`broadphase_sdf`	`zero_tcnt`, `tiled_sdf_filter`	the voxel prefilter over each surviving tile's vertices
21	`scan`	`_scan_top`	the survivors' exclusive scan, its length read on the device
22	`compact_nCa2`	`set_ct_nCa2_tiled`, `scatter_nCa2_tiled`, `build_ul_nCa2_tiled`	the candidate vertices compacted
23	`narrowphase_sdf`	`compute_nsdf`	the spline SDF at each candidate (tricubic, or trilinear by choice)
24	`scan`	`_reduce`, `_scan_top`, `_scan_add`	the penetrating ones' scan, over several blocks
25	`compact_nCa3`	`set_ct_nCa3`, `scatter_nCa3`, `build_ul_nCa3`	the contacts compacted, sorted by (world, segment)
26	`narrowphase_sdf`	`gather_contacts`	each contact's world point, normal and gap
	The contact solve	`cd/driver.py`: `SolverDriver.contact_dynamics`	
27	[`contact_reduction`] `reduce.reduce`	`assign_rows`, `score_contacts`, `mark_winners`	Newton's reduction: at most 240 contacts a segment, kept by their bins' corners and depth

#	call	kernels	what they do
28	[groups] `groups.before_export`	`group_contacts`, `group_count`, `group_assign`, `group_place`, `flip`	each grouped contact's bin, weight and carried frame; the groups and their members; the frames' buffers swapped
29	`contact_dynamics`	`export_lodos` [`export_groups`], `gather_M`	Lodos's contacts [and groups] appended to `d.contact`; M from Lodos's tree blocks into MJWarp's sparse `d.M`
30	`constraint.make_constraint`	`_zero_constraint_counts`, `_equality_connect`, `_equality_weld`, `_equality_joint`, `_equality_tendon`, `_friction_dof`, `_friction_tendon`, `_limit_ball`, `_limit_slide_hinge`, `_limit_tendon`, `_efc_contact_init`, `_efc_contact_jac_sparse`, `_efc_contact_update` [`_contact_group_sums`]	the constraint rows: equalities, friction loss and limits (each kernel launches whether or not there are rows of its kind); then the contacts' rows, their Jacobian and their weights and references
31	`nlsp.solve_prox`, before the rounds	`_solve_init_dof`, `_prox_jaref_kernel`, `_prox_init`, `_fill_contact_impratio`, `_solve_init_efc`, `_solve_init_jaref_kernel`, `mul_m`, `_update_constraint_efc`, `_zero_qfrc_constraint_sparse`, `_update_constraint_init_qfrc_constraint_sparse`, `_update_gradient_zero_grad_dot`, `_update_gradient_grad`, `_update_gradient_init_h_sparse`, `_JTDACJ_sparse`, `_padding_h`, `_update_gradient_cholesky_blocked`	$\ddot{q}$ from the warm start; $J\ddot{q} - a_{\text{ref}}$; each contact's anchor, Baumgarte bias and de Saxcé term; the worlds' solver state reset; $M\ddot{q}$; the rows' states and the constraint force; the gradient; the Hessian $M + J^\top DJ$ with the elliptic cone's curvature, M densified and the rows' terms added; its Cholesky factor and the first direction
32	round 1: Newton, under `capture_while`	`mul_m`, `_linesearch_jv_fused_kernel`, `_linesearch_iterative_kernel`, the update, gradient and Hessian kernels of row 31, `_solve_done`	a Newton iteration of every unconverged world: J times the direction, the exact line search, the new state, gradient, Hessian and direction, the convergence test; repeated on the device until every world is done
33	round 1: the prox update	`_prox_zero_residuals`, `_prox_update`, `_prox_check`, `_prox_solve_init_efc`, `_prox_jaref_kernel`, and the update, gradient and Hessian kernels	new anchors and de Saxcé terms from the converged forces; each world's fixed-point test; the next round's Newton state
34	rounds 2–10, each under `capture_if`	as rows 32 and 33	skipped once every world's fixed point holds: `prox_iters_per_qp` (10) rounds in all
35	after the rounds	`_prox_cache_fill`, CUB's radix sort	the converged anchors into the cross-step cache, sorted by key: MJWarp's contacts only, as Lodos's are `NOCARRY`
	Integration	`rbd/integrate.py`: `integrate`	
36	[implicitfast] `implicit.correct`	`_implicit_qacc`, `_free_bodies`	MuJoCo's form: $(M - hD)^{-1}M\ddot{q}$ per tree, a lone free body by its own 6×6 with its gyroscopic derivative
37	`symplectic_euler`	`_integrate_vel`, `_integrate_pos`, `_advance_time`	$\dot{q}$ += $h\ddot{q}$; q from the new $\dot{q}$, quaternions by the exponential map; the clock

Two things launch outside the step. `make_sim` builds every context once, with its own kernels and host work (hull tables, the map from Lodos's M blocks to MJWarp's `d.M`). The viewer captures `forward`, the integrator and the whole step as three graphs, `forward`'s and the step's with the kernels of `lodos.utils.calibrate.Tracker`, which keep the largest demand on every capacity; a recorded step runs `forward`, reads a frame on the host (`viewer/collect.py`), then runs the integrator. It launches no kernel of its own.

Table 2: What options and models add to Table 1, or replace in it. Launch counts are as captured.

option or model	kernels
`mode="mj"`	MJWarp's own Newton solve (`nlsp.py`: `_solve`) replaces rows 31–35: row 31's kernels without the `_prox` ones, then one `capture_while` of row 32; no rounds, no cache. 101 launches on `arm_box_6`.
`row_space`	`rowspace.prepare` once a solve: `_rows_kernel` ($Z = L^{-1}J^\top$, with Lodos's per-tree factor), `_gram_rows` ($Z^\top Z$). In place of the Hessian and its Cholesky, each Newton system by `_forward`, `_stage_kernel`, `_system`, `_cholesky_kernel`, `_scatter`, `_lift`, `_back`, `_finish`: Woodbury over the rows.
`shared_hessian`	`_shared_hessian`, a block of 512 threads a world, sums the Hessian in shared memory in place of `_update_gradient_init_h_sparse` and `_JTDACJ_sparse`, wherever they launch. By default (`None`) on where it can be built from 64 padded dofs, a model of 48 dofs or more: on `sharpa_5_sdf` (140), not on `arm_box_6` (36).
`fused_newton` ($n_v \leq 16$)	`fused.solve`: `newton_kernel`, a block a world, replaces rows 31–34, every round included, but for the four of row 31 that set the rows up, `_fill_contact_impratio`, `_solve_init_dof`, `_prox_jaref_kernel` and `_prox_init`, which launch before it; row 35 follows it. 67 launches on `incline_sdf`, its `njmax` cut to 960 so that a world's rows fit in shared memory.
`contact_reduction`, `groups`	rows 27 and 28, `export_groups` and `_contact_group_sums`.
a dense model ($n_v \leq 32$)	`_efc_contact_jac_dense`, `_update_constraint_init_qfrc_constraint_dense`; the Hessian by `_update_gradient_JTDAJ_dense_tiled` and, for an elliptic cone, `_update_gradient_JTCJ_dense`; `_update_gradient_cholesky` on one tile.
trees of 13 dofs or more	the bracketed kernels of rows 3, 4 and 11; with actuators and sites, rows 5 and 9 and `_site_poses`. 375 launches on `sharpa_5_sdf`, with its shared Hessian.
`integrator="implicitfast"`	rows 12 and 36.

3 The source files

Table 3: `lodos/`, the simulator.

file	what it holds
`sim.py`	`make_sim`, every context of a scene built once; `Surfaces`, `Sim`, `default_pair_params`; the impedance rule
`step/__init__.py`	`forward`, `step_sim`, `make_step_graph`
`rbd/types.py`	the rigid-body model and state (`make_rbd_context`); refuses what it does not model
`rbd/forward.py`	`fwd_rbd`, rows 1–12 of Table 1; actuation
`rbd/smooth.py`	kinematics, centres of mass, M and its factor (serial, or tiled for large trees), velocities, Newton–Euler, transmissions, solves with M
`rbd/passive.py`, `integrate.py`, `implicit.py`, `io.py`	springs, damping and gravity compensation; semi-implicit Euler; implicitfast; states in and out without breaking a graph
`col/sdf/types.py`	Lodos collision's model and data, every capacity and option (`make_collision_context`)
`col/sdf/collide.py`	the pipeline's stages (rows 18–26); counts, contacts, overflow and capacity tools
`col/sdf/broadphase.py`, `compact.py`, `narrowphase.py`	the pose, box, tile and voxel-filter kernels; the compaction between stages; the SDF at the candidates and the contact gather
`col/sdf/__init__.py`	surface data: `SurfaceData`, `PlaneSurface`, `load_surface_data`, `load_surface_file`, `scale_surface`, `write_surface_obj`
`col/sdf/spline/c2.py`, `c0.py`, `implicit.py`	the tricubic (C2 field, C1 normals) and trilinear (C0 field, normals discontinuous across cell faces) SDFs, and a bake's per-surface evaluator
`col/sdf/io.py`, `serialize.py`	making surfaces: quantizing, Hilbert or Morton vertex order (the `mesh` extra, loaded on first use)
`col/prim/driver.py`	`PrimCollision`: MJWarp's collision on Lodos's poses (rows 13–17)

file	what it holds
`col/prim/collision_driver.py`, `collision_primitive.py`, `collision_convex.py`, `collision_gjk.py`, `collision_core.py`, `types.py`	MJWarp's collision, adapted: its broadphase (row 14); its primitive narrowphase (row 15); its convex one, GJK and EPA, which the hybrid replaces for pairs of hulls and whose types it reuses; the contacts' parameters and writes; the types and constants they share
`col/prim/hybrid/`	the exact hull narrowphase: `pipeline.py` its host side, `kernels.py` its stages, `ccd.py` GJK and EPA on Gauss-map cell supports, `epa_warp.py` EPA a warp a pair, `meshdata.py` the hulls' topology, `truth.py` float64 depths, to adjudicate a pair by hand
`cd/driver.py`	`SolverDriver`, the solve over every contact (rows 27–35); `resolve_options`, `overrides`; the modes; the exact mode's Baumgarte slop; the shared Hessian's default
`cd/solver/constraint.py`, `nlsp.py`	MJWarp's constraint solve, adapted: the constraint rows (`make_constraint`, row 30); nonlinear least squares with proximal minimization, MJWarp's Newton solve with the proximal loop around it (`solve`, rows 31–35)
`cd/solver/block_cholesky.py`, `io.py`, `types.py`	the blocked Cholesky factorization the Newton solve and `row_space` use; the contact groups' arrays; the solver's contexts and the constraints' types and states
`cd/params.py`	the pairs' 13-column parameters; the spring-damper law that weighs group members
`cd/reduce.py`, `groups.py`, `bins.py`	contact reduction (row 27); contact groups (row 28), and their normal bins and moment sectors
`cd/rowspace.py`, `fused.py`	the Newton system through the rows (`row_space`); the whole solve as one kernel a world (`fused_newton`)
`mjcf/types.py`, `io.py`	MJWarp's model and data on the device, adapted, which its collision and the solve share: `Model`, `Data`, `Option`, the contact types (`GROUP`, `NOCARRY`); `put_model`, `put_data`, `override_model`, the capacity estimates
`utils/math.py`, `kernels.py`, `scan.py`	quaternion and spatial algebra, MuJoCo's `MINVAL`, and MJWarp's helpers; the kernel cache and timed scopes MJWarp's code uses; the scan whose length lives on the device (rows 21 and 24)
`utils/states.py`, `calibrate.py`, `tuning.py`	state sets, the starts the tuning tools run from; the demand on every capacity (`Tracker`); tuning files, their fingerprints and staleness

Table 4: `viewer/`, the live viewer, its scenes and their assets.

file	what it holds
`__main__.py`, `app.py`	the command line; the window, ImGui panels on Newton's OpenGL viewer
`session.py`	a session: a scene's builds per configuration, its takes, the sims it ran under; the options' speed arrows (`FASTER`)
`collect.py`, `trajectory.py`	a frame read off a one-world `Sim` on the host; a recorded run, frames on the host
`draw.py`, `geometry.py`	the drawing (`TrajectoryViewer`); both pipelines' geometry as one scene
`joints.py`, `performance.py`	editing a state joint by joint; the Performance panel
`pairs.py`	the Collision pairs folder: friction, solref and solimp per pair, and a Lodos pair one- or two-sided
`newton_link.py`	Newton's sims, MuJoCo Warp and Kamino, in a process of their own, for comparison
`newton/`	that process's side, a uv project of its own (Newton 1.7.0) that imports no `lodos`: `newton_scene.py`, a scene in Newton; `runner.py`, which answers `newton_link`
`scenes/catalog.py`	the panel's scenes in order (`PANEL`), those loaded by name (`BUILTIN`), annotated MJCF files
`scenes/assets.py`	where the shipped assets are; shipped surfaces by name (`load_surface`)
`scenes/drop.py`, `representations.py`, `armadillo.py`, `mixed.py`	stacked drops of armadillos or cubes, as surfaces or polytopes, on a floor or walled; the armadillo drop; both pipelines in one pile
`scenes/sharpa.py`, `shadow.py`	SHARPA and Shadow hand piles, five in a stack or twenty in four
`scenes/sharpa_armadillo.py`, `shadow_kong.py`	the SHARPA hand holding an armadillo; the Shadow hand's grasp of the Kong toy
`scenes/fbf.py`, `nutbolt.py`, `nist.py`, `franka.py`	the FBF paper's exactness scenes; Newton's nut-bolt; Isaac Lab's NIST parts, board, driven nuts and threading; the Franka
`scenes/data/`, `assets/`	recorded controls and starts; the scenes' meshes, surfaces, hulls, MJCFs and licences

3.1 Benchmarks and tools

- `bench/`: `lodos_mjwarp/` and `lodos_kamino/`, the batched-throughput comparisons of Lodos's mj mode with MuJoCo Warp and of its exact mode with Kamino, each as Newton runs it, and `lodos_setting.py`, how both set Lodos up
- `tools/performance_tuning/`: `tune.py`, one scene end to end, and `tune_all.py`; its steps `explore.py` (state sets), `calibrate.py` (capacities), `tile_tuner.py` (tiles), with `sweep.py`, `time_variant.py`, `profile_scene.py` and `common.py`; `summarize.py`, which writes `tuned/README.md`; `migrate_formats.py`, which brings files of an older stored format to the current one; `tuned/` and `tuned/mj/`, every scene's tuning file under the exact and the mj mode
- `tools/mesh_processing/`: the pipeline from an MJCF to baked surfaces, `run_pipeline.py`: `fuse_meshes_by_body.py`, `calibrate_cp_decomp.py`, `interactive_remesh.py`, `fix_nonmanifold_meshes.py`, `create_surface_data.py` (the bake); `pack_spheres.py`, a baked surface's inner spheres; their helpers `mjcf.py`, `topology.py`, `instant_meshes.py`, `pamo_driver.py`, `pamo_worker.py`, `mesh_viewer.py`; `scene_surfaces.py` (boxes and spheres); `nutbolt/`, `nist/` (those parts' surfaces)
- `notebooks/`, `docs/`: five tutorial notebooks, the fifth making a surface from a mesh; the manuals and guides, their LaTeX in `docs/manual/`